Stellar Classification (GALAXY / QSO / STAR)
PMLDL Projects 2026 — Technical Report

Overview. This report presents an end-to-end solution for the stellar classification task: predicting whether an observed object is a GALAXY, QSO, or STAR from 14 photometric and astrometric features. We benchmark three existing solutions (***Logistic Regression***, ***Random Forest***, ***CatBoost***), propose a custom neural ***Tabular ResNet*** selected among three tested architectures, improve it via hyperparameter tuning, and compare all five models under a strict protocol in which the test split is used exactly once. ***CatBoost*** achieves the best test balanced accuracy (0.9739); ***Random Forest*** the best macro F1 (0.9749); the tuned ***Tabular ResNet*** reaches 0.9718 balanced accuracy and the best QSO F1 of all models (0.9530).

1. Problem Statement

1.1 Task

Astronomical object classification from photometric observations: given measurement features of an object recorded by the Sloan Digital Sky Survey (SDSS), predict its class — GALAXY, QSO (quasar), or STAR. This is a multiclass supervised classification task.

1.2 Data source

We use the public SDSS17 dataset ("Stellar Classification Dataset — SDSS17", Kaggle, provider: fedesoriano), 100,000 objects derived from SDSS DR17. Features: celestial coordinates (alpha, delta), five photometric magnitudes (u, g, r, i, z), redshift, and six engineered color indices (u_g, g_r, r_i, i_z, u_r, g_i) — 14 numerical features in total. Nine identifier/observation metadata columns are removed during preparation.

Data quality note: the raw photometric bands u, g, z contain −9999 sentinel values for objects with missing photometry. They were retained and propagate into the engineered color features as extreme values (observed around −10017); the median imputer does not treat them as missing. All models are affected identically under the same feature pipeline, so model comparisons remain valid; the implications are discussed in Section 7.2.

The data is split into train / validation / test (70,000 / 15,000 / 15,000) with a stratified 70/15/15 split, seed 42, stratified by the target variable.

Class	Train	Val	Test
GALAXY	41,611	8,917	8,917
QSO	13,273	2,844	2,844
STAR	15,116	3,239	3,239

In relative terms, GALAXY is ≈59.4%, STAR ≈21.6%, and QSO ≈19.0% of every split.

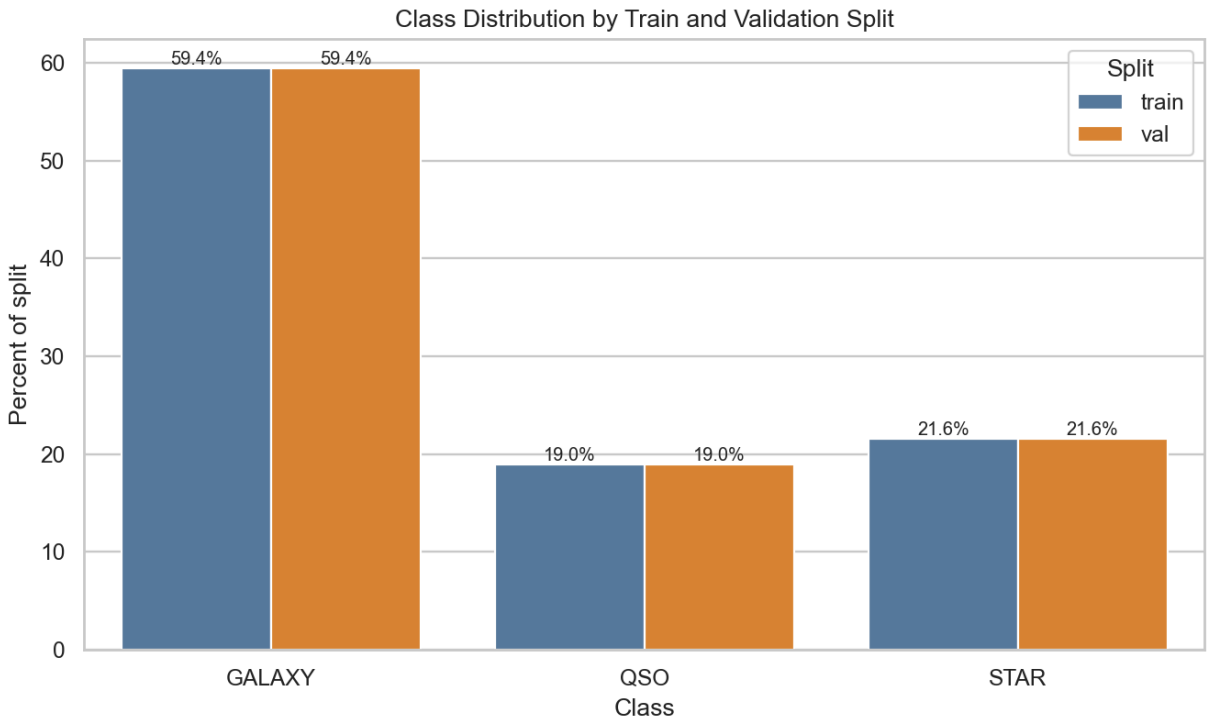


Figure 1. Class distribution across the train, validation, and test splits.

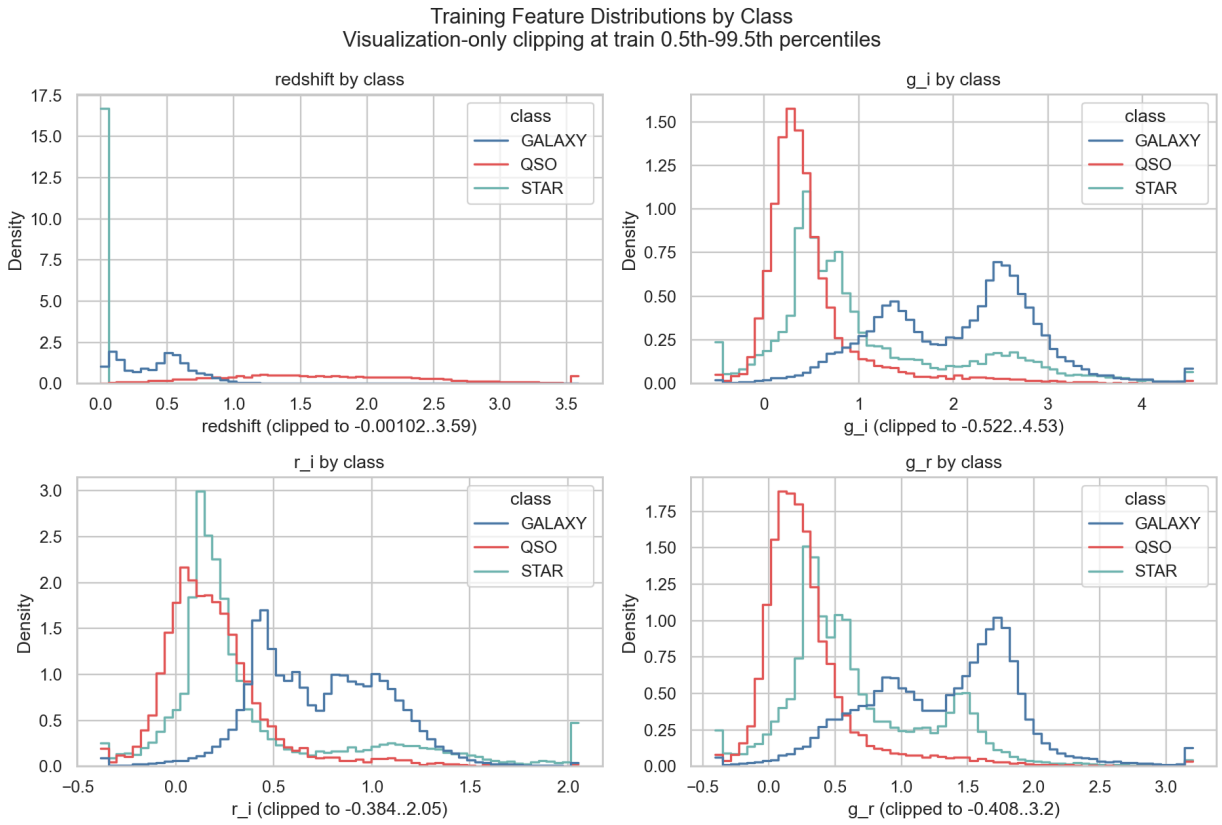


Figure 2. Distributions of the most informative features by class (visualization-only clipping at the train-set 0.5th/99.5th percentiles; original values are preserved for model training).

1.3 Evaluation metrics

The classes are moderately imbalanced, so ordinary accuracy is insufficient as the sole selection metric: a model favoring the majority GALAXY class can score well while missing the minority QSO class.

Primary metric: balanced accuracy — the unweighted mean of per-class recall, so every class contributes equally despite imbalance. This is also the evaluation metric of the closely related Kaggle Playground stellar-classification competition.

Secondary metric: macro F1 — computes F1 independently for each class with equal class weights, additionally accounting for precision; it rejects models that inflate recall at the cost of false positives.

Every model reports: *accuracy; macro precision; macro recall; macro F1; weighted F1; balanced accuracy; per-class precision, recall, F1, and support; and a confusion matrix*. Metric calls use zero_division=0 where applicable. ROC-AUC (one-vs-rest) is reported as a supplementary diagnostic for the final test evaluation. The test split is used exactly once, after all model selection decisions are completed (Section 5).

1.4 Scope

The project satisfies the three minimal requirements as follows: **(1)** existing solutions are tested and compared with metrics — Section 2; **(2)** a new custom model is proposed and evaluated — Section 3; **(3)** the proposed model is modified via architecture trials and hyperparameter tuning, with the metric effect reported — Section 4. The final one-time test comparison of all models is given in Section 5. Section 6 documents reproducibility, and Section 7 concludes.

2. Baselines

2.1 Protocol

Three established model families are benchmarked, following public solutions for the SDSS17 task as precedent (citations in 2.5). All baselines use the same prepared features and label mapping (GALAXY→0, QSO→1, STAR→2), are fitted on train.csv only, and are evaluated on val.csv only. No hyperparameter optimization is performed at this stage. Preprocessing (median imputation, standardization where applicable) is fitted inside the estimators on training data only; both target columns (class, class_encoded) are removed from the features to prevent leakage.

Leakage prevention checks: neither target representation remains in the features; train and validation feature columns and order match; all selected features are numeric; missing and infinite values are checked; imputation and scaling are fitted only inside estimators trained on train.csv. No SMOTE or global resampling is used.

Features used by every model: alpha, delta, u, g, r, i, z, redshift, u_g, g_r, r_i, i_z, u_r, g_i.

2.2 Models

Logistic Regression — interpretable linear baseline: median imputation, StandardScaler, class_weight="balanced", max_iter=3000, seed 42.

Random Forest — nonlinear bagging baseline: 300 trees, class_weight="balanced_subsample", max_features="sqrt", seed 42, n_jobs=-1, conservative defaults otherwise.

CatBoost — strong gradient-boosting baseline: MultiClass loss, 500 iterations, learning rate 0.05, depth 6, l2_leaf_reg=3, auto_class_weights="Balanced", random_seed=42, CPU, no verbose output.

2.3 Validation results

Model	Balanced Accuracy	Macro F1	Accuracy	Macro Precision	Macro Recall	Weighted F1	Fit Time (s)	Prediction Time (s)
CatBoost	0.9724	0.9686	0.9723	0.9650	0.9724	0.9723	7.5298	0.0172
Random Forest	0.9697	0.9733	0.9771	0.9772	0.9697	0.9769	11.3982	0.1347
Logistic Regression	0.9477	0.9345	0.9409	0.9230	0.9477	0.9411	1.0182	0.0062

Fit and prediction times were measured during the validation benchmark run on the development machine; timing is not used for model selection.

Table 3. Per-class validation results

Model	Class	Precision	Recall	F1	Support
Logistic Regression	GALAXY	0.9719	0.9277	0.9493	8,917
Logistic Regression	QSO	0.8784	0.9167	0.8971	2,844
Logistic Regression	STAR	0.9188	0.9988	0.9571	3,239
Random Forest	GALAXY	0.9757	0.9860	0.9808	8,917
Random Forest	QSO	0.9655	0.9241	0.9443	2,844
Random Forest	STAR	0.9905	0.9991	0.9948	3,239
CatBoost	GALAXY	0.9827	0.9705	0.9766	8,917
CatBoost	QSO	0.9354	0.9466	0.9409	2,844
CatBoost	STAR	0.9768	1.0000	0.9883	3,239

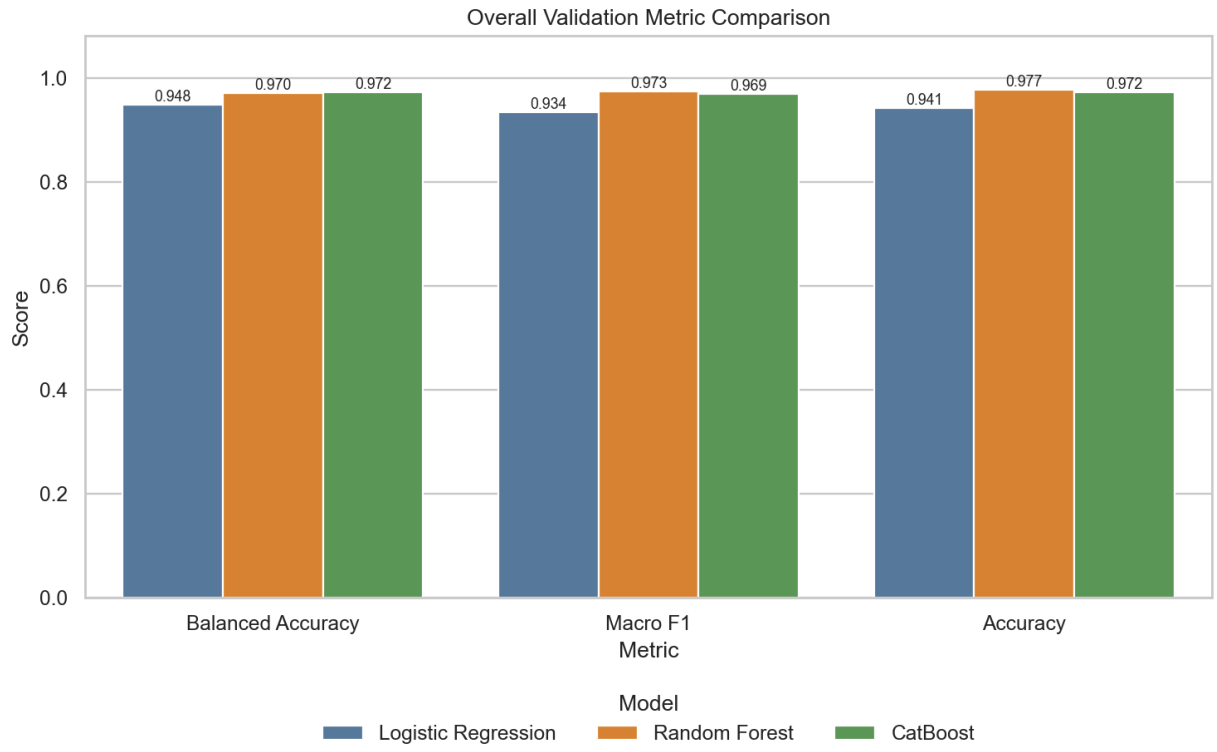


Figure 3. Overall metric comparison of the baselines on validation.

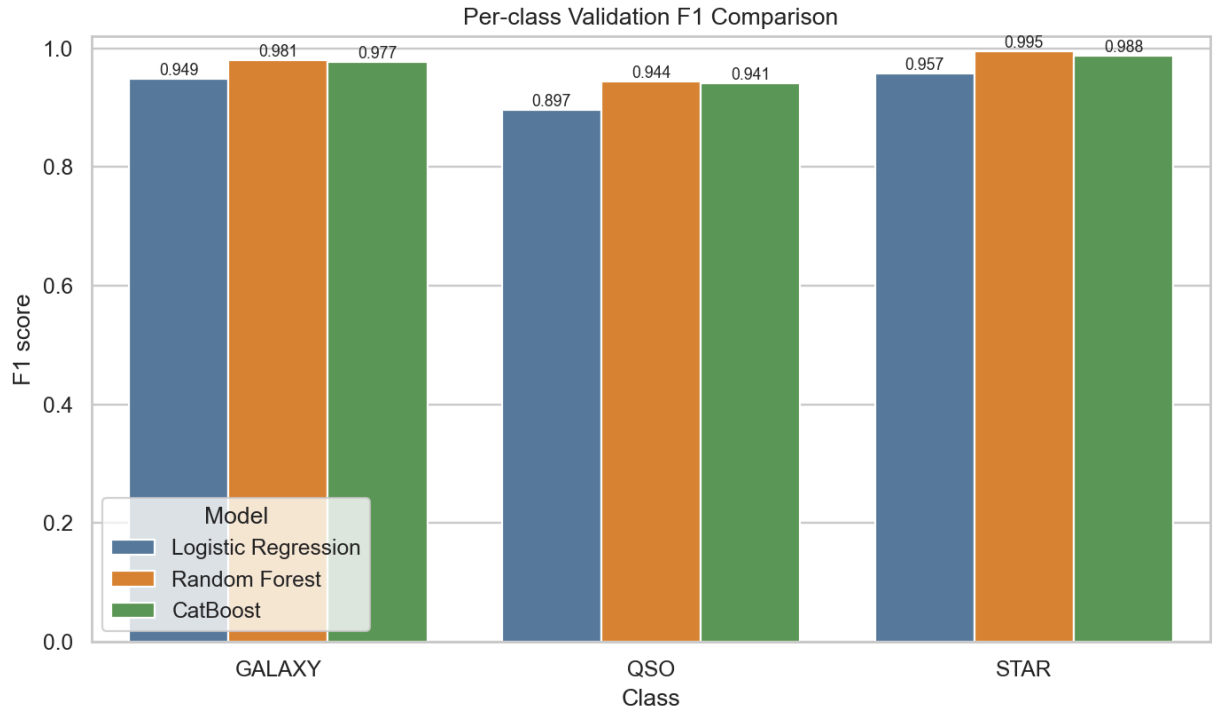


Figure 4. Per-class F1 comparison of the baselines on validation.

2.4 Findings

CatBoost achieves the best validation balanced accuracy (0.9724), driven partly by stronger recall for the minority QSO class; ***Random Forest*** achieves the best macro F1 (0.9733) and accuracy (0.9771), indicating a better overall precision-recall balance. ***Logistic Regression*** is clearly weakest on every metric, showing that linear decision boundaries are insufficient for this feature space.

For all baselines the weakest class is QSO, and the dominant confusion is between GALAXY and QSO; STAR is nearly separable (***CatBoost*** reaches STAR recall 1.000). The most common off-diagonal errors are: ***CatBoost*** — true GALAXY predicted as QSO (186 rows, 2.09% of true GALAXY); ***Random Forest*** — true QSO predicted as GALAXY (216 rows, 7.59% of true QSO); ***Logistic Regression*** — true GALAXY predicted as QSO (359 rows, 4.03% of true GALAXY). These results define the benchmark that the proposed model must be compared against.

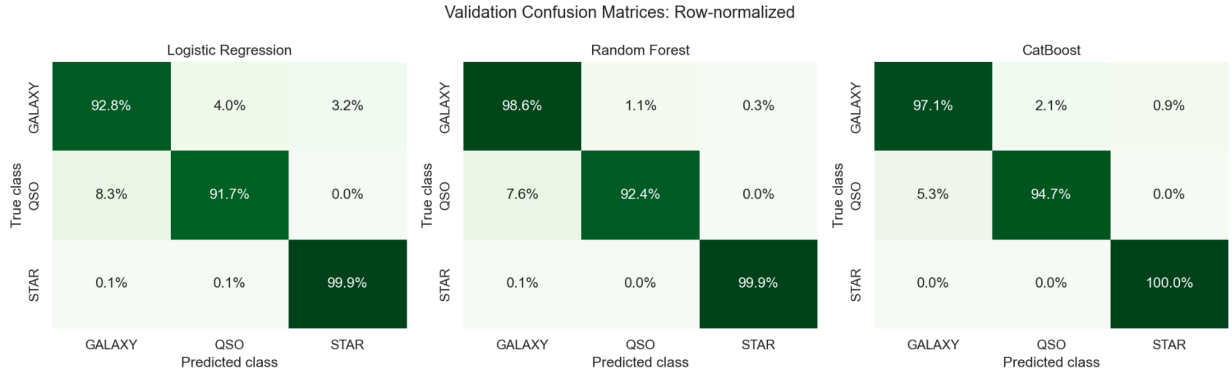


Figure 5. Validation confusion matrices of the three baselines (individual matrices: confusion_matrix_logistic_regression.png, confusion_matrix_random_forest.png, confusion_matrix_catboost.png).

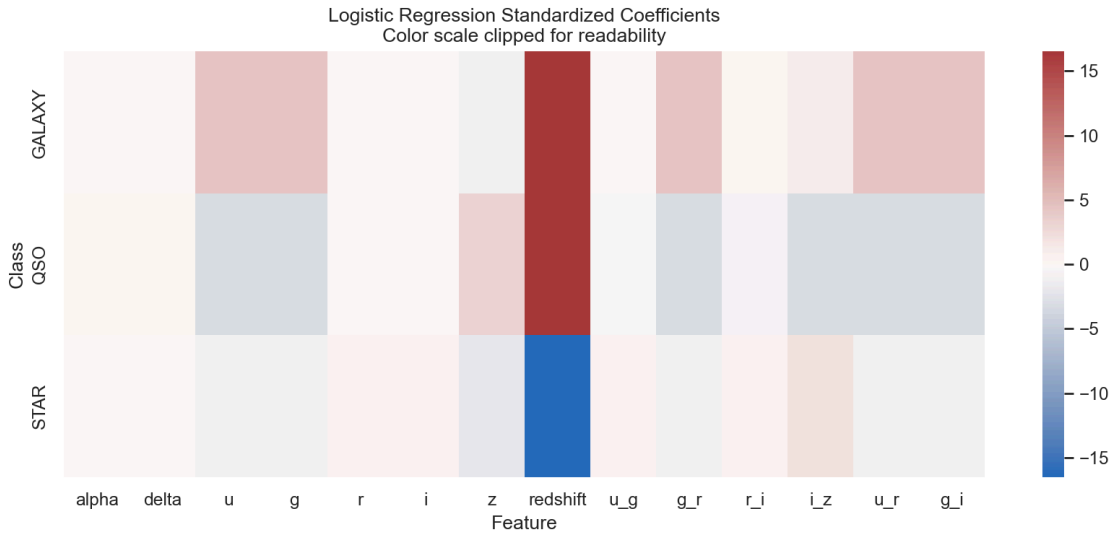


Figure 6. ***Logistic Regression*** coefficients.

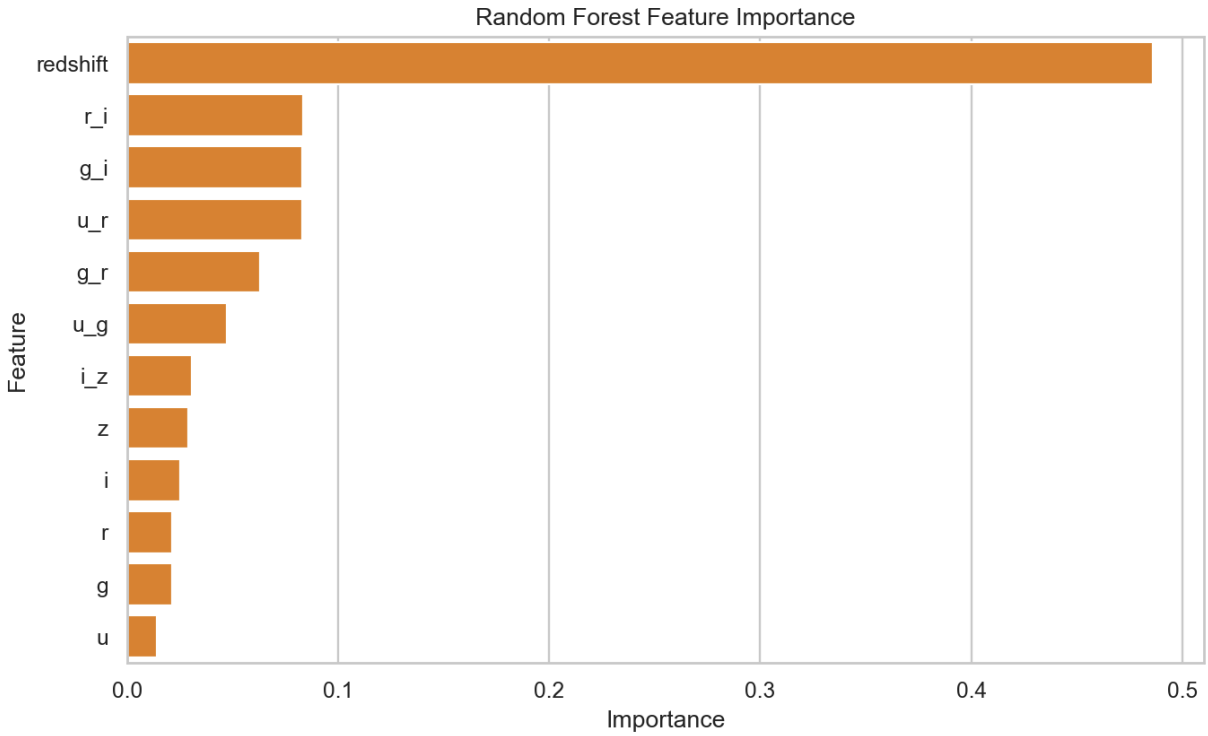


Figure 7. ***Random Forest*** feature importance.

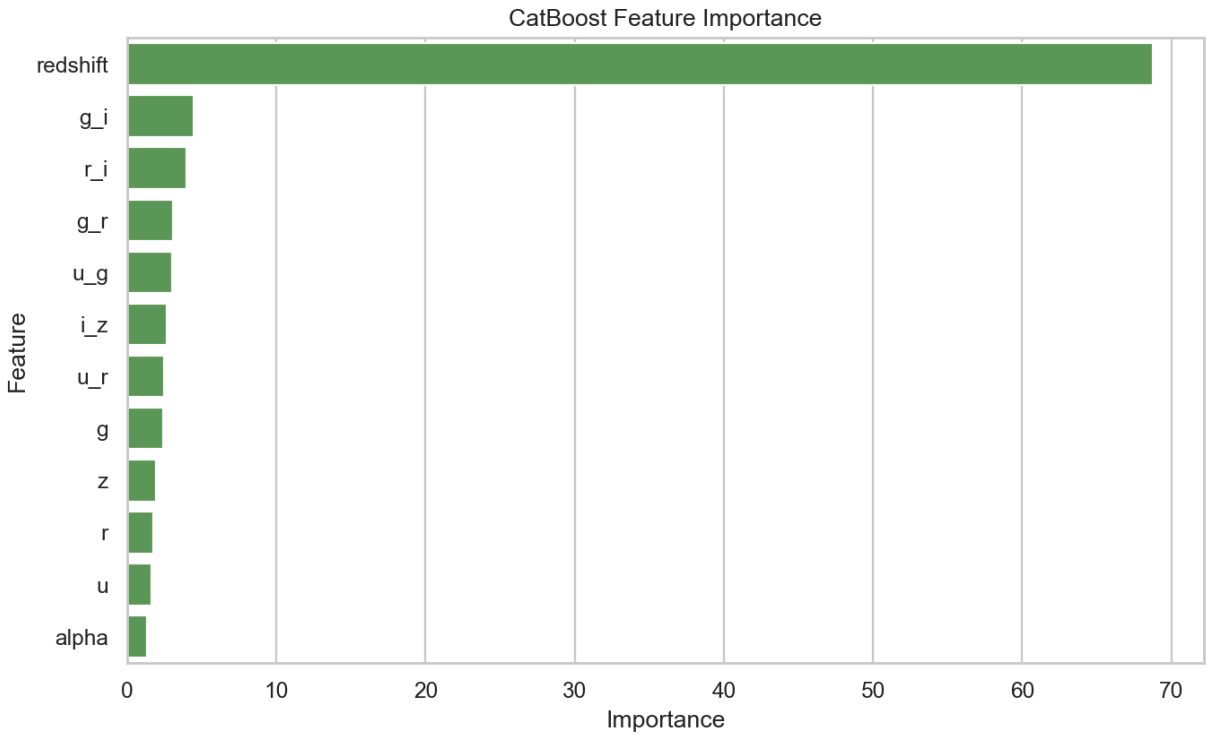


Figure 8. ***CatBoost*** feature importance.

2.5 Public solution citations

Logistic Regression and Random Forest project (GitHub, wonhee3472): https://github.com/wonhee3472/Stellar_Classification — precedent for comparing Logistic Regression and Random Forest on SDSS stellar classes.

Random Forest notebook (Kaggle, leyalamammadzada): https://www.kaggle.com/code/leylamammadzada/stellar-classification-random-forest — precedent for a Random Forest stellar-classification baseline.

CatBoost notebook (Kaggle, aerdem): https://www.kaggle.com/code/aerdem/stellar-classification-catboost-97-accuracy — precedent for a CatBoost multiclass baseline.

Original SDSS17 dataset (Kaggle, fedesoriano): https://www.kaggle.com/datasets/fedesoriano/stellar-classification-dataset-sdss17

Published scores from these sources are not directly comparable with ours (different splits, features and preprocessing); this benchmark reproduces the model families under one standardized evaluation protocol.

3. Proposed Model

3.1 Model selection and motivation

Three custom neural architectures were investigated for this project: an FT-Transformer, a Hybrid FT-Transformer, and a ***Tabular ResNet***. Their observed validation balanced accuracies were 0.9688, 0.9658, and 0.9694, respectively. ***Tabular ResNet*** was selected as the proposed model because it achieved the best validation balanced accuracy among the tested custom neural architectures. It is appropriate for low-dimensional numerical tabular data, and its residual connections support deeper MLP transformations while maintaining stable optimization. It is also computationally simpler than the Transformer alternatives used in the experiments.

The strongest baseline remains CatBoost, with validation balanced accuracy of approximately 0.9724. Therefore, the selected proposed model does not yet outperform the strongest baseline.

3.2 Input representation

The model consumes 14 numerical input features. Median imputation and StandardScaler preprocessing are fitted exclusively on train.csv. The fitted objects are then only applied to val.csv during validation. This train-only fitting prevents validation leakage.

3.3 Architecture

TabularResNetClassifier (implemented in src/model.py, covered by unit tests in tests/test_model.py) receives an input tensor of shape [B, 14]. An input projection, Linear(14, 256), produces a 256-dimensional representation. The representation then passes through four residual blocks. Each block applies BatchNorm1d(256), Linear(256, 512), GELU, Dropout(0.1), Linear(512, 256), and Dropout(0.1), followed by residual addition with the block input.

After the four residual blocks, the classification head applies BatchNorm1d(256), GELU, and Linear(256, 3). The output is raw logits for three classes. No softmax is applied inside the model because CrossEntropyLoss expects raw logits.

3.4 Training procedure

The model is trained from scratch with random seed 42, batch size 256, and 30 epochs. The optimizer is AdamW with learning rate 1e-3 and weight decay 1e-4; the loss function is CrossEntropyLoss. No pretrained weights are used, and no fine-tuning stage is part of this initial proposed-model experiment. The separate model-improvement stage is documented in Section 4.

Artifacts of this run: checkpoint models/tabular_resnet.pt, training history reports/training/tabular_resnet_history.csv, and loss / balanced-accuracy / macro-F1 curves under reports/figures/.

3.5 Validation protocol

train.csv is used for training, while val.csv is used for model selection and evaluation. test.csv is not accessed during model development. The best checkpoint is selected by validation balanced accuracy. Balanced accuracy is the primary metric, and macro F1 is the secondary metric.

3.6 Initial results

Table 4. Validation balanced accuracy of all evaluated models

Model	Validation Balanced Accuracy
Logistic Regression	0.9477
Random Forest	0.9697
CatBoost	**0.9724**
FT-Transformer	0.9688
Hybrid FT-Transformer	0.9658
Tabular ResNet	0.9694

Tabular ResNet is the strongest tested custom neural architecture in the initial proposed-model experiment. It does not yet exceed the strongest ***CatBoost*** baseline. The follow-up tuning changes are documented in Section 4.

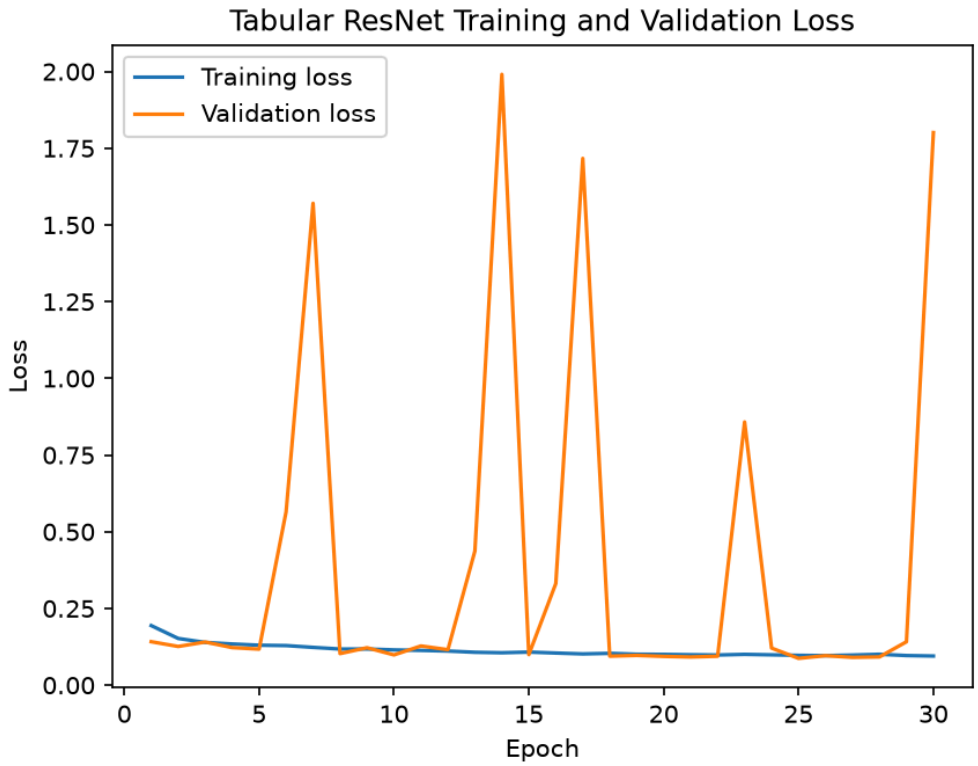


Figure 9. Untuned Tabular ResNet: validation and training loss per epoch (best epoch 21).

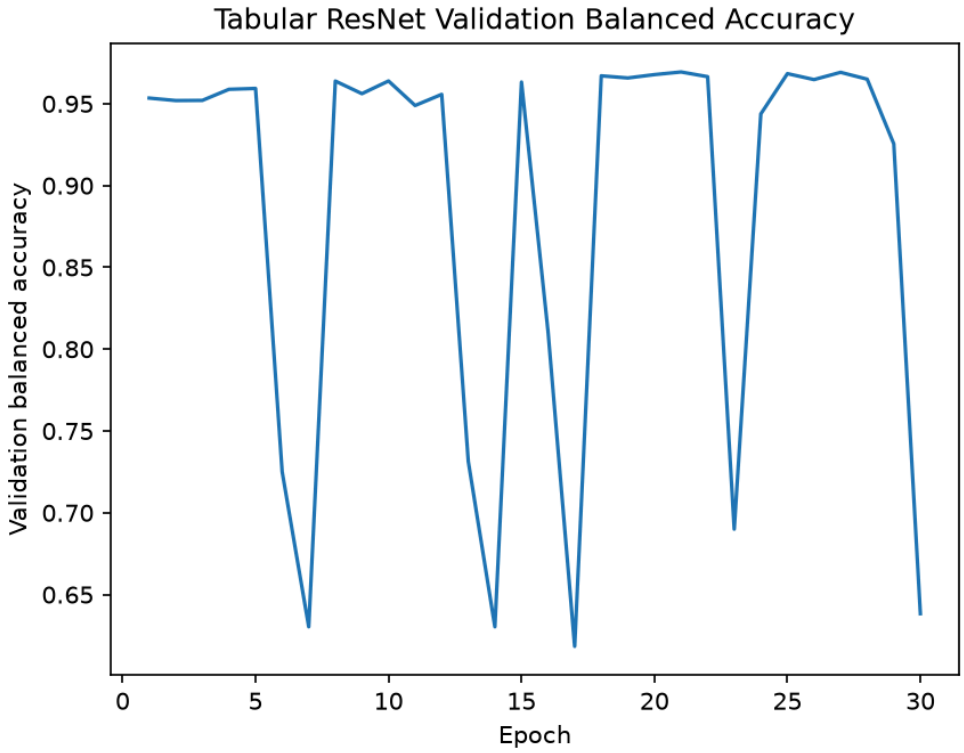


Figure 10. Untuned Tabular ResNet: validation balanced accuracy per epoch.

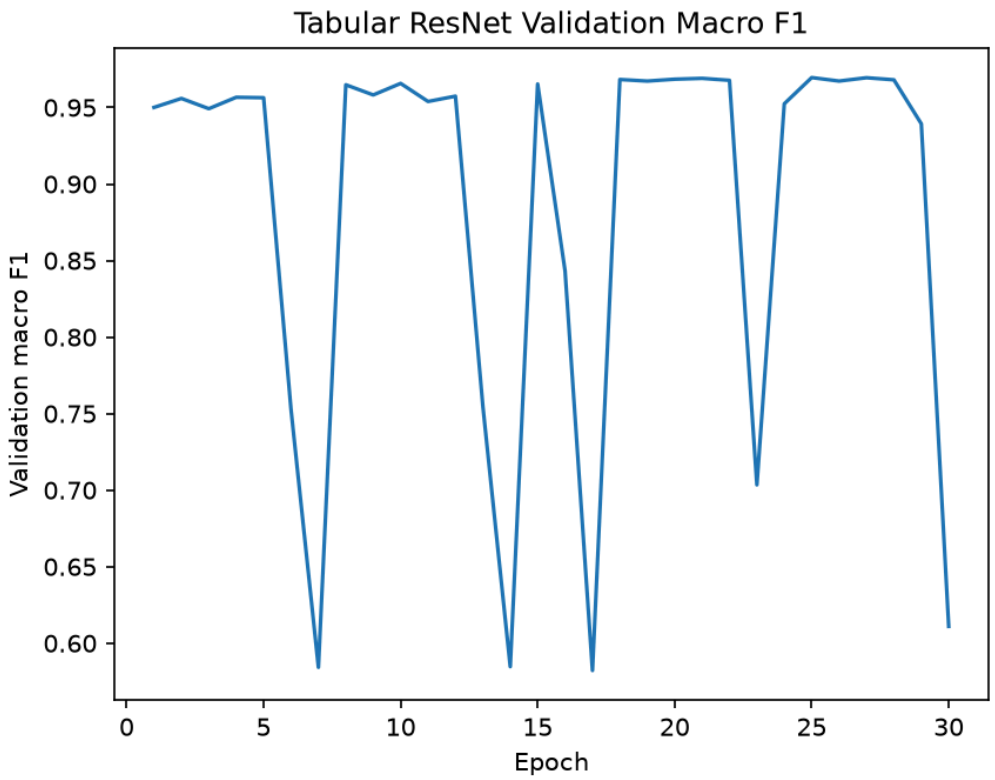


Figure 11. Untuned Tabular ResNet: validation macro F1 per epoch.

3.7 Limitations

The initial architecture was evaluated under one fixed training configuration. Only validation metrics are reported at this stage, while test.csv remains untouched. Final conclusions require the tuned validation run (Section 4) and the one-time final test evaluation (Section 5).

4. Model Improvements

4.1 Improvement target

The selected model for this stage is TabularResNetClassifier. The previous untuned reference used a fixed residual MLP configuration:

```
batch size: 256
epochs: 30
learning rate: 1e-3
weight decay: 1e-4
hidden dimension: 256
residual block hidden dimension: 512
residual blocks: 4
dropout: 0.1
```

```
loss: unweighted CrossEntropyLoss
```

The objective of the improvement stage was to modify and optimize this selected model without accessing test.csv. The validation split remains the only model selection split.

4.2 Tuning decision

CatBoost and ***Tabular ResNet*** have different roles in this project. ***CatBoost*** is treated as a strong existing tabular-learning baseline, while Tabular ResNet is the selected custom proposed model required by the project specification. For this reason, the model-improvement stage tunes Tabular ResNet rather than relabeling ***CatBoost*** as the proposed model. ***CatBoost*** remains the strongest overall validation model, but the custom-model requirement and the model-improvement requirement are evaluated through the Tabular ResNet experiments.

The improvement stage tested architecture and loss-function changes first:

- input LayerNorm before the first projection;
- a wider hidden representation;
- an additional hidden layer in the classification head;
- train-only class-weighted CrossEntropyLoss.

This variant reached only 0.9560 validation balanced accuracy, so it was not kept as the final tuned model. The final improvement therefore keeps the selected residual-MLP architecture and optimizes its hyperparameters and training schedule. This satisfies the model-improvement requirement through hyperparameter tuning while avoiding a worse architecture change.

That rejected variant was a historical manual trial. Only its summary value survives; no checkpoint, full history, or executable configuration was retained, so the value is not part of the authoritative metrics and cannot be independently verified. The final tuned experiment is fully represented by code, history, and checkpoint.

4.3 Tuned configuration

The training script uses an explicit TrainingConfig in src/train_model.py for the tuned run:

```
batch size: 256
maximum epochs: 45
learning rate: 7e-4
```

```
weight decay: 1e-4
hidden dimension: 256
residual block hidden dimension: 512
residual blocks: 4
dropout: 0.05
gradient clipping norm: 1.0
learning-rate scheduler: ReduceLROnPlateau, factor 0.5, patience 4
early stopping patience: 100 epochs, so the 45-epoch tuned run is not
stopped early
minimum balanced-accuracy improvement: 1e-4
```

The changes address the instability observed in the previous training history: late epochs showed sharp validation degradation, so the tuned pipeline now saves only validation-balanced-accuracy improvements, reduces the learning rate on a validation plateau, clips gradients, and uses lower dropout than the untuned reference.

4.4 Validation effect

The tuned run was evaluated on val.csv only. test.csv was not loaded during training or tuning.

Table 5. Untuned vs. tuned ***Tabular ResNet*** (validation)

Model	Best epoch	Validation balanced accuracy	Macro F1	Validation loss
Untuned Tabular ResNet	21	0.9693998068	0.9690497756	0.0917610324
Tuned Tabular ResNet	45	0.9694004393	0.9704551329	0.0831951720

The tuned model is only marginally better by the primary metric: balanced accuracy improves by 0.0000006325. The improvement is therefore not practically meaningful by balanced accuracy alone. However, the tuned model is better on the supporting metrics: macro F1 improves by 0.0014053573, and validation loss decreases by 0.0085658604.

The tuned neural model still does not outperform the strongest validation baseline, ***CatBoost***, which has validation balanced accuracy 0.9723533082.

The final tuned configuration was selected because it was the best validated ***Tabular ResNet*** variant among the checked improvement attempts, not because it surpassed the strongest non-neural baseline. The report therefore avoids claiming a substantial primary-metric gain and instead records the actual outcome: balanced accuracy stayed almost unchanged, while macro F1 and validation loss improved.

The main interpretation is that the original ***Tabular ResNet*** was already close to its validation optimum for this feature set. The tuning stage still has practical value because it:

- tested a more aggressive architecture/loss modification and rejected it after validation performance dropped;
- preserved the primary metric instead of degrading it;
- improved macro F1, which is the supporting metric used to check class-level precision-recall balance;
- reduced validation loss, indicating better-calibrated validation logits under the same train/validation protocol;
- kept test.csv untouched, so the final test evaluation remains unbiased.

Therefore, the tuned model is described as a limited improvement, not as a substantial performance breakthrough. The result supports the conclusion that further large gains are more likely to come from the stronger tabular baseline family or ensembling than from simple ***Tabular ResNet*** hyperparameter changes. The generalization of these gains to the test split is verified in Section 5

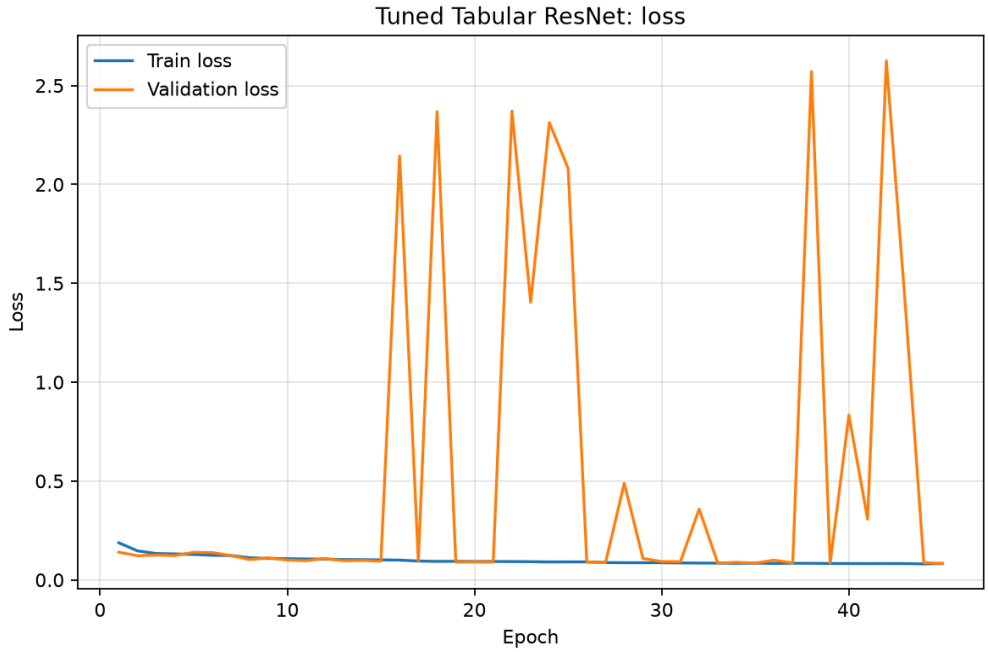


Figure 12. Tuned ***Tabular ResNet***: validation and training loss per epoch (best epoch 45).

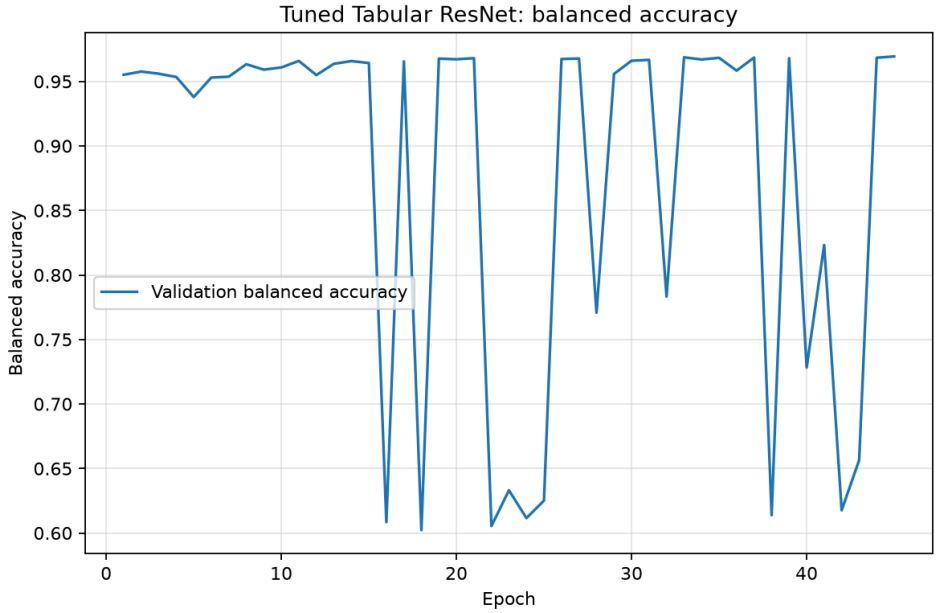


Figure 13. Tuned ***Tabular ResNet***: validation balanced accuracy per epoch.

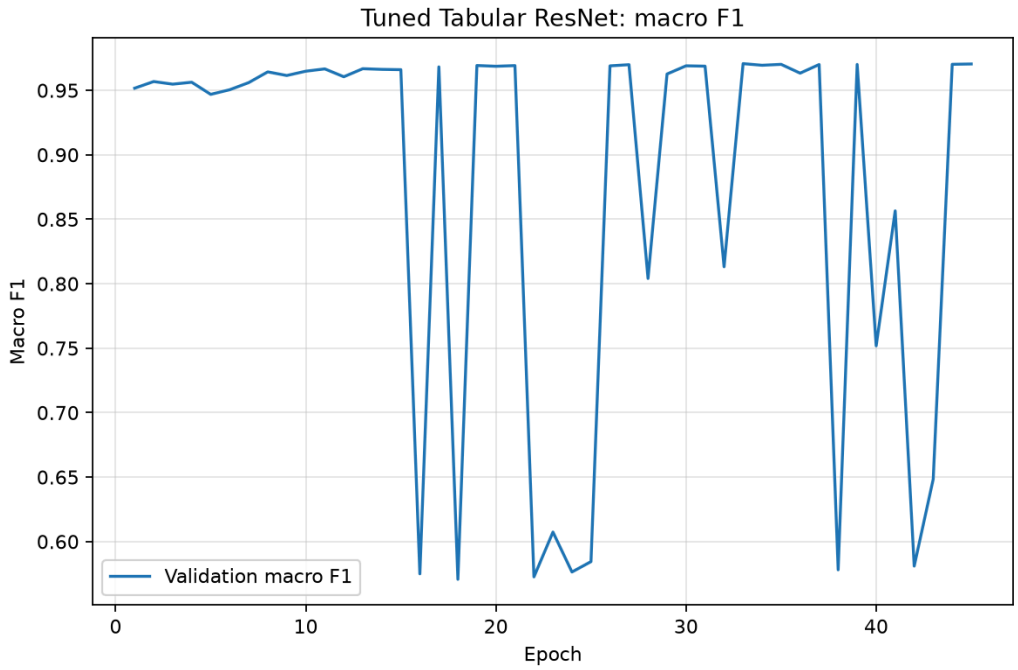


Figure 14. Tuned ***Tabular ResNet***: validation macro F1 per epoch.

4.5 Outputs of the tuned run

The tuned stage writes separate artifacts instead of overwriting the untuned reference:

```
models/tabular_resnet_tuned.pt
reports/training/tabular_resnet_tuned_history.csv
reports/figures/tabular_resnet_tuned_loss.png
reports/figures/tabular_resnet_tuned_balanced_accuracy.png
reports/figures/tabular_resnet_tuned_macro_f1.png
```

The tuned stage itself did not access test.csv; its one-time evaluation is reported in Section 5.

5. Results

5.1 Evaluation protocol

All five models — three baselines (***Logistic Regression***, ***Random Forest***, ***CatBoost***), the proposed untuned ***Tabular ResNet***, and the tuned ***Tabular ResNet*** — were evaluated on the held-out test split (15,000 objects) for the first and only time, after all model selection and hyperparameter tuning had been completed on the validation split. The test split was not loaded or inspected at any earlier stage of the project.

The evaluation was executed once by src/evaluate_test.py under the same protocol as the validation benchmark: baseline estimators are rebuilt by the same build_models() function with the same fixed seeds (so they are identical to the validation models), neural checkpoints are restored from the files saved during Sections 3–4, and the neural preprocessing (median imputation, standardization) is fitted on the training split only. All metrics are frozen in reports/tables/test_metrics.csv and reports/test_results.json; a marker file blocks accidental re-runs of the test evaluation.

Environment note. The one-time test evaluation was executed once on a host environment with Python 3.14.7 and NumPy 2.5.2; all other package versions match Table 8 (PyTorch 2.14.0, CatBoost 1.2.10, pandas 3.0.5, scikit-learn 1.9.0, Matplotlib 3.11.1, seaborn 0.13.2). The full environment record is preserved in reports/test_results.json. The canonical Docker workflow (Section 6) verifies the validation benchmark and both neural checkpoints; the test evaluation is intentionally not repeated, so the test split is used exactly once.

5.2 Overall comparison

Models are ranked by the primary metric (balanced accuracy) on the test split.

Table 6. Final test-set comparison of all models

Model	Type	Val BA	Test BA	Test Macro F1	Test Accuracy	Test ROC-AUC (OvR)
CatBoost	Baseline	0.9724	**0.9739**	0.9682	0.9718	**0.9964**
Random Forest	Baseline	0.9697	0.9729	**0.9749**	**0.9783**	0.9963
Tabular ResNet (tuned)	Improved	0.9694	0.9718	0.9716	0.9750	0.9954
Tabular ResNet (untuned)	Proposed	0.9694	0.9714	0.9689	0.9726	0.9944
Logistic Regression	Baseline	0.9477	0.9489	0.9354	0.9418	0.9874

5.3 Per-class results

Table 7. Test F1 per class (full precision/recall/support tables: reports/tables/test_per_class_metrics.csv)

Model	GALAXY F1	QSO F1	STAR F1
Logistic Regression	0.9505	0.9030	0.9528
Random Forest	**0.9818**	0.9502	**0.9926**
CatBoost	0.9762	0.9432	0.9851
Tabular ResNet (untuned)	0.9772	0.9496	0.9798
Tabular ResNet (tuned)	0.9792	**0.9530**	0.9825

5.4 Analysis

Model ranking is stable between validation and test. The ordering by the primary metric is preserved: ***CatBoost*** leads, ***Random Forest*** is second, both ***Tabular ResNet*** variants follow, and ***Logistic Regression*** trails. Every validation-to-test gap is small and positive (at most +0.0032 for ***Random Forest***), which indicates that the test split is slightly easier than validation but shows no sign of validation overfitting or selection bias during development.

Baselines. ***CatBoost*** confirms the best validation balanced accuracy on the test split (0.9739), and ***Random Forest*** confirms the best macro F1 (0.9749) and accuracy (0.9783), exactly as on validation. ***Logistic Regression*** remains clearly weakest on every metric, confirming that linear decision boundaries are insufficient for this feature space. All four non-linear models achieve ROC-AUC above 0.994, so their probabilistic rankings are excellent and the remaining errors are boundary cases between classes.

Proposed model. The untuned ***Tabular ResNet*** (test BA 0.9714) is competitive with the strongest baselines and improves on ***Logistic Regression*** by a large margin, but

does not surpass ***CatBoost*** by the primary metric, consistent with the validation outcome.

Improvement effect generalizes to the test split. The tuned configuration improves the untuned ResNet on test: balanced accuracy 0.9718 vs. 0.9714 and macro F1 0.9716 vs. 0.9689 (+0.0027, larger than the +0.0014 observed on validation). The tuned model also achieves the best QSO F1 (0.9530) of all five models, including both strongest baselines. The macro F1 gain observed during tuning was therefore not a validation artifact.

Error structure. QSO is the weakest class for every model, and the dominant confusion is between GALAXY and QSO, consistent with the validation confusion matrices and with the physics of the task: low-redshift quasars overlap with galaxies in photometric color space, while STAR is nearly separable (***CatBoost*** reaches STAR recall 1.000). This matches the interpretation in Section 2 that balanced accuracy and macro F1 are the right metrics: a model biased toward the majority GALAXY class would score well on accuracy but degrade on QSO.

Runtime. Baseline fit times are 0.7 s (***Logistic Regression***), 8.4 s (Random Forest) and 5.3 s (CatBoost) on CPU; the ***Tabular ResNet*** fit time is reported as zero because the evaluation loads the checkpoints trained in Sections 3–4 (45 epochs; see Section 4 for training details).

6. Reproducibility

6.1 Canonical environment

The canonical environment is Docker using python:3.11.0-slim-bullseye pinned to manifest digest sha256:b5ee5360b357dcd0ab2e1cc5e22fa589b6734bbd55776df6b1700994febf1527, Python 3.11.0 on linux/amd64, and CPU execution. requirements.txt pins every direct Python dependency with ==, while requirements.lock.txt pins the complete resolved environment. The Docker build runs pip check, and the entrypoint verifies Python and every locked distribution before work. Docker is the source of truth; no host Python installation is used. The fixed architecture selects widely available Linux wheels and consistent numerical behavior; Docker Desktop provides emulation on non-amd64 hosts.

Table 8. Verified direct dependency versions in the canonical environment

Package	Version
Python	3.11.0

CatBoost	1.2.10
certifi	2026.7.22
Matplotlib	3.11.1
NumPy	2.4.6
pandas	3.0.5
pytest	9.1.1
scikit-learn	1.9.0
seaborn	0.13.2
tabulate	0.10.0
PyTorch	2.14.0

6.2 Randomness and determinism

The project-wide seed is 42. src/reproducibility.py controls Python random, PYTHONHASHSEED, NumPy, PyTorch CPU and CUDA seeds, deterministic PyTorch algorithms, cuDNN settings, and the DataLoader generator/worker seeds. The split, scikit-learn estimators, and ***CatBoost*** also use seed 42. The canonical container fixes numerical thread counts to one and uses CPU only.

The saved neural checkpoints were historically trained on Apple MPS. Their reported validation predictions and metrics have been verified on CPU, but a fresh CPU retraining is not claimed to be bitwise identical to the historical MPS optimization trajectory. The canonical run therefore retrains deterministic baselines and verifies neural checkpoints. Every reported metric, including ***Random Forest***, is compared with strict absolute tolerance 1e-12 in the canonical linux/amd64 container. Timing columns are excluded because wall-clock time is not a scientific result.

A non-canonical native arm64 run produced a 0.0005435868 ***Random Forest*** balanced-accuracy difference from the stored historical result. This observed platform drift is documented but is not accepted by the canonical verifier and does not justify a relaxed Docker tolerance. The clean linux/amd64 Docker run reproduced the historical ***Random Forest*** metrics exactly.

6.3 Data access

The source is version 1 of the public "Stellar Classification Dataset — SDSS17," maintained on Kaggle by fedesoriano and derived from Sloan Digital Sky Survey DR17. The canonical script downloads the versioned archive from the official Kaggle API without credentials. It fails clearly if the resource is unavailable or if the extracted CSV SHA-256 differs from faefcad9b950309282024dd351289ee14ae30f65a7b2ef32c9fa442da9dbe585. All generated hashes are recorded in reports/reference_metrics.json; the expected checksums are listed in Table 9. No private drive or local path is used, and no credentials are required.

Table 9. Expected SHA-256 checksums

File	SHA-256
star_classification.csv	faefcad9b950309282024dd351289ee14ae30f65a7b2ef32c9fa442da9dbe585
train.csv	f0659a57c02afc18d7860c886233a30719b43d5cdc28615307b0161a4c665d21
val.csv	aa84702f24b2cf1b72f54aaac489a2090748dce064737f7a5d3a0abf5c855c6d
test.csv	33a4c0a9a103b9ced0804103bd45bb02a1bf72d24487d785ad1ad376dd9a80a0

6.4 Data preparation

src/prepare_data.py reproduces the original notebook procedure. It removes nine identifier/observation metadata columns, creates u_g, g_r, r_i, i_z, u_r, and g_i colour differences, maps GALAXY/QSO/STAR to 0/1/2, and makes stratified 70/15/15 train/validation/test splits with seed 42. The result contains 70,000/15,000/15,000 rows and 14 numerical model features. All three generated files must match their recorded SHA-256 values (Table 9).

To recreate the prepared splits from the raw Kaggle file, place star_classification.csv in data/ and run python -m src.prepare_data; the canonical workflow performs the download and preparation automatically (Section 6.5).

6.5 Docker reproduction

Run exactly:

```
docker compose up --build --exit-code-from reproduce
```

Compose propagates the reproduction service exit code. The workflow installs dependencies, downloads and prepares data, runs tests, retrains baselines, evaluates saved neural checkpoints, verifies metrics, audits anonymity, and writes reproduction_outputs/verified_metrics.json on the host.

6.6 Reproducing baselines

The canonical workflow runs python -m src.baseline_benchmark. ***Logistic Regression***, ***Random Forest***, and ***CatBoost*** are fitted on train.csv and evaluated on val.csv. Stable metrics appear in reports/tables/baseline_metrics.csv and the canonical consolidated JSON output. Figures and per-class metrics are regenerated in reports/ inside the container. On Windows, py -m src.baseline_benchmark can be used if python points to the Microsoft Store launcher. An executed notebook presenting the same benchmark workflow is included in notebooks/baseline_benchmark.ipynb.

6.7 Reproducing the proposed model

The proposed ***Tabular ResNet*** training implementation is src/train_model.py. The historical proposed checkpoint is models/tabular_resnet.pt; the canonical workflow checks its hash and reevaluates it on the recreated validation split. This is the reproducible verification path for the reported historical result. To run the initial 30-epoch CPU training configuration, use:

```
docker compose run --rm reproduce python -m src.train_model --experiment
initial
```

6.8 Reproducing the improved/tuned model

The tuned configuration in src/train_model.py uses learning rate 7e-4, dropout 0.05, gradient clipping, and ReduceLROnPlateau. To launch a fresh, deterministic CPU tuned run in the container, use:

```
docker compose run --rm reproduce python -m src.train_model --experiment
tuned
```

The canonical one-command workflow instead verifies the selected checkpoint models/tabular_resnet_tuned.pt, because the published artifact was trained on MPS and fresh CPU optimization is not expected to be bitwise identical.

6.9 Expected results

Table 10. Expected validation results of the canonical run (full precision)

Experiment	Validation balanced accuracy	Validation macro F1	Validation accuracy
Logistic Regression	0.9476993243584669	0.9344956819867626	0.9409333333333333
Random Forest	0.9697020845241635	0.9732969664359739	0.9770666666666666
CatBoost	0.9723533081902743	0.9685890288006633	0.9723333333333334
Tabular ResNet	0.9693998067762958	0.9690497755767767	0.9728666666666667
Tuned Tabular ResNet	0.9694004392953532	0.9704551328887030	0.9742000000000000

These are the validation results used for model selection. The reserved test split was evaluated exactly once after selection was completed; the test results are reported in Section 5.

6.10 Output artifacts

The reviewer-facing machine-readable output is reproduction_outputs/verified_metrics.json. Baseline tables and figures are regenerated under reports/ in the container. Neural training writes its checkpoint to models/tabular_resnet_tuned.pt, history to reports/training/, and plots to reports/figures/ inside the run container.

6.11 Validation status and known limitations

The canonical command was executed on a clean host installation and completed with exit code 0. It verified Python 3.11.0, all 38 locked packages, raw and split

checksums, 40 passing tests, exact baseline metrics, both neural checkpoints, and the anonymity audit. The complete record is in reports/final_validation.md (included in the source code).

The canonical run requires outbound HTTPS access to Kaggle. It fails rather than silently continuing if the pinned public resource changes or becomes unavailable. Native non-canonical arm64 execution may exhibit the ***Random Forest*** drift described above. Historical FT-Transformer, Hybrid FT-Transformer, and rejected wider/class-weighted variant values have incomplete machine-readable training histories; they remain contextual architecture trials, not authoritative final metrics.

7. Conclusion

7.1 Summary of findings

We built an end-to-end classification pipeline for the SDSS17 stellar classification task (GALAXY / QSO / STAR, 100,000 objects, 14 features including six engineered color indices) under a strict evaluation protocol: a stratified 70/15/15 split, balanced accuracy as the primary metric, macro F1 as the supporting metric, and a test split used exactly once, after all model selection decisions were made.

Three baselines were benchmarked under one standardized protocol with public solutions cited as precedent. ***CatBoost*** achieved the best validation balanced accuracy (0.9724) and ***Random Forest*** the best macro F1 (0.9733). As the proposed model, we implemented a custom ***Tabular ResNet*** and selected it among three custom neural architectures (FT-Transformer 0.9688, Hybrid FT-Transformer 0.9658, ***Tabular ResNet*** 0.9694 validation balanced accuracy). The improvement stage tuned the ResNet training schedule (lower learning rate, ReduceLROnPlateau, gradient clipping, reduced dropout) and preserved balanced accuracy while improving macro F1 and validation loss; a more aggressive architecture/loss variant was tested and rejected after validation performance dropped.

On the single final test evaluation, the ranking matched validation: ***CatBoost*** was best by balanced accuracy (0.9739), ***Random Forest*** best by macro F1 (0.9749) and accuracy (0.9783). The tuned ***Tabular ResNet*** reached 0.9718 balanced accuracy and 0.9716 macro F1, confirming the tuning gains on unseen data and achieving the best QSO F1 (0.9530) of all models on the hardest class. All validation-to-test gaps were below 0.004, indicating no overfitting to the validation split.

The neural model does not outperform gradient boosting on the primary metric for this feature set; with balanced accuracy already near 0.97, the remaining errors concentrate on the GALAXY/QSO boundary, where photometric features alone are least separable.

7.2 Limitations

Sentinel values. The raw photometric bands u, g, z contain −9999 sentinels for objects with missing photometry. They were retained and propagate into the engineered color features as extreme values (observed around −10017); the median imputer does not treat them as missing. All models are affected identically under the same feature pipeline, so model comparisons remain valid, but absolute accuracy may slightly understate the attainable level.

Single split. Results are reported on one train/validation/test split without confidence intervals or significance testing. Differences below roughly 0.003 (e.g., ***CatBoost*** vs. ***Random Forest***) should be treated as not statistically meaningful.

Marginal tuning gain on the primary metric. The tuned ResNet improves macro F1 and validation loss but is essentially unchanged by balanced accuracy, and it does not surpass the strongest baseline by the primary metric.

Limited hyperparameter search. The tuning stage covered a small, hand-picked configuration set rather than an exhaustive search, so conclusions about the ResNet's ceiling are preliminary.

Class imbalance. Imbalance (QSO ≈ 19%) is addressed through the metric choice and class weighting in the baselines; no resampling or cost-sensitive training was adopted for the proposed model after the weighted-loss variant degraded validation performance.

7.3 Future work

Clean the photometric sentinels and regenerate the splits; tune ***CatBoost*** and other gradient-boosting variants; study ensembling of tree and neural models, which make different types of errors on the GALAXY/QSO boundary; broaden neural hyperparameter optimization; add probability calibration and per-class threshold analysis; and validate on newer SDSS data releases to test distribution drift.

References

Gorishniy, Y., Rubachev, I., Khrulkov, V., Babenko, A. "Revisiting Deep Learning Models for Tabular Data", NeurIPS 2021. https://arxiv.org/abs/2106.11959

Dataset provider, fedesoriano, "Stellar Classification Dataset — SDSS17": https://www.kaggle.com/datasets/fedesoriano/stellar-classification-dataset-sdss17

Sloan Digital Sky Survey DR17: https://www.sdss.org/dr17/

Kaggle API documentation: https://www.kaggle.com/docs/api

PyTorch CrossEntropyLoss: https://docs.pytorch.org/docs/stable/generated/torch.nn.CrossEntropyLoss.html

PyTorch AdamW:
https://docs.pytorch.org/docs/stable/generated/torch.optim.AdamW.html
PyTorch ReduceLROnPlateau:
https://docs.pytorch.org/docs/stable/generated/torch.optim.lr_scheduler.ReduceLROnPlateau.html
PyTorch gradient clipping:
https://docs.pytorch.org/docs/stable/generated/torch.nn.utils.clip_grad_norm_.html
PyTorch torch.save: https://docs.pytorch.org/docs/stable/generated/torch.save.html
PyTorch BatchNorm1d:
https://docs.pytorch.org/docs/stable/generated/torch.nn.BatchNorm1d.html
PyTorch Linear: https://docs.pytorch.org/docs/stable/generated/torch.nn.Linear.html
PyTorch reproducibility notes:
https://docs.pytorch.org/docs/stable/notes/randomness.html
scikit-learn StandardScaler:
https://scikit-learn.org/stable/modules/generated/sklearn.preprocessing.StandardScaler.html
scikit-learn SimpleImputer:
https://scikit-learn.org/stable/modules/generated/sklearn.impute.SimpleImputer.html
scikit-learn balanced_accuracy_score:
https://scikit-learn.org/stable/modules/generated/sklearn.metrics.balanced_accuracy_score.html
scikit-learn f1_score:
https://scikit-learn.org/stable/modules/generated/sklearn.metrics.f1_score.html
pandas read_csv: https://pandas.pydata.org/docs/reference/api/pandas.read_csv.html
Matplotlib pyplot.plot:
https://matplotlib.org/stable/api/_as_gen/matplotlib.pyplot.plot.html
Docker Compose file reference: https://docs.docker.com/compose/compose-file/